

Java Backend Architecture

Interview Series

Chapter 14.5

Message Queues & Event Streaming

50+ Interview Q&A

Code Examples

Architecture Diagrams

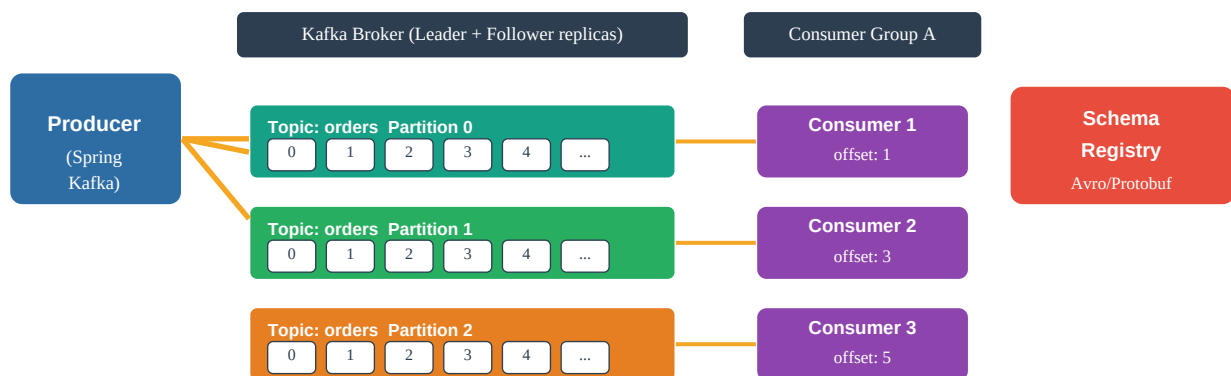
Advanced Topics

Chapter 14.5 – Message Queues & Event Streaming

Overview

Message queues and event streaming platforms are fundamental to building scalable, decoupled distributed systems. Apache Kafka dominates event streaming with its log-based architecture. RabbitMQ excels at task queues with flexible routing. AWS SQS provides managed queuing for cloud-native applications. Understanding their trade-offs drives critical architecture decisions for senior Java backend engineers.

Apache Kafka Architecture



Kafka: Topics → Partitions → Offsets → Consumer Groups (1 partition per consumer max)

Kafka Core Concepts

Topic	Logical channel for messages. Multiple producers write; multiple consumer groups read independently. Topic has a replication factor (copies across brokers).
Partition	Physical unit of parallelism. Topic split into N partitions; each partition ordered log. More partitions = higher parallel throughput. Partitioning key determines which partition a message goes to.
Offset	Sequential message ID within a partition. Consumer tracks its offset. Replay: reset offset to beginning. Exactly-once: commit offset only after processing.
Consumer Group	Group of consumers that share consumption of a topic. Each partition assigned to exactly 1 consumer in group. Adding consumers up to N (partition count) increases parallelism. N+1 consumers: one is idle.
Log Compaction	Retains only the latest message per key. Removes superseded versions. Used for: CDC changelog topics, materialized views. "Infinite" retention for latest state without unbounded growth.
Schema Registry	Confluent Schema Registry stores Avro/Protobuf/JSON schemas. Producer registers schema; consumer verifies. Schema evolution: backward/forward compatible changes enforced. Prevents incompatible schema deployments.

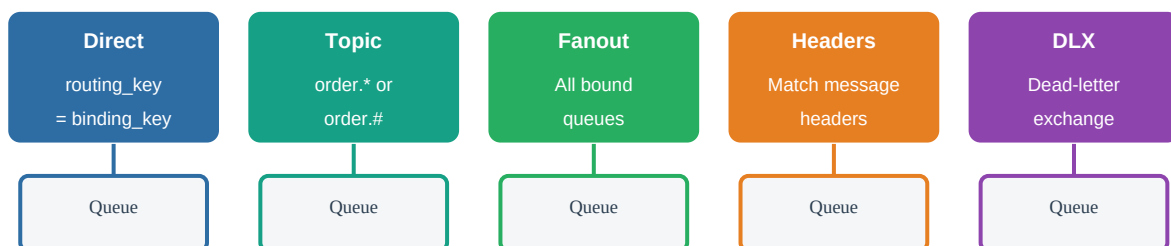
Kafka Exactly-Once Semantics

- Idempotent producer: each producer gets unique ID; broker deduplicates by (producer_id, sequence_number)
- Transactions: atomic multi-partition writes — all partitions commit or none do
- Transactional consumer: only read committed messages (isolation.level = read_committed)
- End-to-end exactly-once: producer transactions + idempotent consumer with offset commit in same transaction

```
// Spring Kafka exactly-once producer @Bean public ProducerFactory<String, Object>
producerFactory() { Map<String, Object> props = new HashMap<>();
props.put(ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, true);
props.put(ProducerConfig.TRANSACTIONAL_ID_CONFIG, "tx-producer-1");
props.put(ProducerConfig.ACKS_CONFIG, "all"); // required for idempotence return new
DefaultKafkaProducerFactory<>(props); } // Kafka Streams exactly-once processing
props.put(StreamsConfig.PROCESSING_GUARANTEE_CONFIG, StreamsConfig.EXACTLY_ONCE_V2); // uses
transactions internally
```

RabbitMQ Architecture

RabbitMQ Exchange Types: **Producer** → **Exchange** → **Binding** → **Queue** → **Consumer**



AMQP Protocol | DLX handles failed/expired messages | Priority Queues supported

Direct Exchange	Routes message to queues whose binding key exactly matches the routing key. Used for: point-to-point task distribution, worker queues.
Topic Exchange	Wildcard routing: * matches one word, # matches zero or more. order.placed → order.*. All orders → order.#. Flexible pub/sub routing.
Fanout Exchange	Routes message to ALL bound queues, ignoring routing key. Used for: broadcast notifications, event multicast to multiple services.
Headers Exchange	Routes based on message header attributes instead of routing key. More flexible, slower. x-match: any (any header matches) or all (all must match).
DLX (Dead-Letter Exchange)	Receives messages that are: rejected with requeue=false, expired (TTL exceeded), queue max-length exceeded. Routes to dead-letter queue for inspection/retry.
Priority Queues	Queues with x-max-priority (1-255). Higher priority messages processed first. Use for: urgent notifications, premium user requests over standard.

RabbitMQ Spring AMQP Example

```
// Spring AMQP configuration @Configuration public class RabbitConfig { @Bean TopicExchange
ordersExchange() { return new TopicExchange("orders.exchange"); } @Bean Queue
orderCreatedQueue() { return QueueBuilder.durable("order.created")
.withArgument("x-dead-letter-exchange", "orders.dlx") .withArgument("x-message-ttl", 300000)
```

```
// 5min TTL .build()); } @Bean Binding binding() { return
BindingBuilder.bind(orderCreatedQueue()) .to(ordersExchange()).with("order.created.#"); } }
RabbitListener(queues = "order.created") public void processOrder(OrderEvent event) { //
Process with at-least-once delivery // Throw exception to NACK and trigger retry/DLX }
```

AWS SQS

Feature	SQS Standard	SQS FIFO
Ordering	Best-effort (not guaranteed)	Strict FIFO (per message group)
Delivery	At-least-once (may deliver duplicate)	Exactly-once within 5-minute window
Throughput	Nearly unlimited	300 TPS (3000 with batching)
Deduplication	Not built-in	ContentBased dedup hash (5 min window)
Message Groups	Not supported	MessageGroupId for parallel FIFO
Visibility Timeout	Default 30s (up to 12h)	Default 30s (up to 12h)
Long Polling	20s max (reduces empty receives)	20s max
DLQ Support	Yes	Yes

Kafka vs RabbitMQ vs SQS Comparison

Feature	Kafka	RabbitMQ	SQS
Protocol	Custom TCP (binary)	AMQP 0-9-1	AWS HTTP API
Model	Log (pull)	Queue (push/pull)	Queue (pull)
Ordering	Per-partition	Per-queue (optional)	FIFO queue only
Delivery	At-least-once / exactl	At-least-once	At-least/exactly-once
Retention	Days/weeks (log)	Until consumed	Up to 14 days
Throughput	Millions/sec	Thousands/sec	Thousands/sec (FIFO: 3)
Use Case	Event streaming, analy	Task queues, routing	AWS-native async

Kafka vs RabbitMQ vs SQS — Feature Comparison

Apache Pulsar

- Multi-tenancy: namespaces → topics (tenant/namespace/topic). Multiple tenants on same cluster
- Tiered storage: automatically offload old data to S3/GCS, brokers serve fresh data
- Stateless brokers: BookKeeper handles storage; brokers are compute-only (scale independently)
- Unified messaging: supports both queuing (exclusive/shared consumer) and streaming (key-shared)
- Geo-replication: built-in cross-region replication with configurable policies

Interview Q&A; — Message Queues & Event Streaming

Q1. What is the difference between a message queue and event streaming?

Message queue (RabbitMQ, SQS): message consumed once by one consumer, then deleted. Designed for task distribution and point-to-point communication. Event streaming (Kafka, Pulsar): events stored in immutable log, retained for days/weeks, multiple consumer groups read independently at their own pace. Streaming supports: replay, audit trail, event sourcing, multiple independently-paced consumers.

Q2. What is a Kafka partition and why does it matter?

A partition is the unit of parallelism in Kafka. A topic with N partitions can be consumed by up to N consumers in parallel within a consumer group. Each partition is an ordered append-only log. Messages with same key always go to same partition (key-based partitioning) — ensures ordering for related events (all events for `user_id=123` are ordered). More partitions = higher throughput but more overhead.

Q3. What is a consumer group in Kafka?

A consumer group is a set of consumers that collectively consume a topic. Each partition is assigned to exactly one consumer in the group — no duplicate processing within group. Adding consumers (up to partition count) increases parallelism. Multiple independent groups can consume same topic without interference — each has its own offset tracking. Used for: one group processes orders for payment, another for inventory, independently.

Q4. What happens when you add a consumer beyond the partition count?

If consumer count > partition count, the excess consumers are idle — no partition to assign. Adding Consumer 4 to a 3-partition topic: C4 is idle until a partition is available. This is why partition count should be set generously upfront (more partitions = more future scale). Reducing consumers: partitions rebalanced to remaining consumers (rebalance protocol). Increasing partitions: possible but may break key-based ordering for existing keys.

Q5. What is log compaction in Kafka?

Log compaction retains only the latest message per key within a partition. Older messages with the same key are deleted during compaction. Result: a compacted topic contains the latest value for every key — like a changelog. Use cases: database changelog (CDC), materialized view updates, configuration/state topics. Enables infinite retention of current state without unbounded disk growth. Tombstone: send null value to delete a key.

Q6. What is the Kafka schema registry?

Confluent Schema Registry stores and versions message schemas (Avro, Protobuf, JSON Schema). Producer fetches schema ID, serializes message with schema ID header. Consumer fetches schema by ID, deserializes. Schema evolution rules: BACKWARD (new schema can read old data), FORWARD (old schema can read new data), FULL (both). Prevents breaking changes: adding required field without default violates backward compatibility — rejected by registry.

Q7. What is Kafka exactly-once semantics and how is it achieved?

Three levels: At-most-once (acks=0, possible loss). At-least-once (acks=all, idempotent consumer, possible duplicates). Exactly-once: (1) Idempotent producer: each message has sequence number, broker deduplicates retries. (2) Transactional API: atomically write to multiple partitions + commit consumer offset. (3) Consumer: `isolation.level=read_committed` — only read committed messages. Enables exactly-once streaming in Kafka Streams.

Q8. What are the 4 RabbitMQ exchange types?

Direct: route to queue with exact routing key match. E.g., "payment.completed" → payment queue. Topic: wildcard routing. "order.*" matches "order.placed", "order.shipped". "order.#" matches any depth. Fanout: broadcast to all bound queues — routing key ignored. Used for: event broadcasting to multiple services. Headers: route by message header key-value pairs. Flexible but slower, rarely used. Default (nameless) exchange: direct with queue name as key.

Q9. What is a Dead-Letter Exchange (DLX) in RabbitMQ?

DLX receives messages that cannot be delivered normally. Messages sent to DLX when: consumer explicitly rejects with `requeue=false`, message TTL expires without consumption, queue max-length exceeded. Configure per-queue via `x-dead-letter-exchange` argument. DLX routes to dead-letter queue for: inspection, alerting, manual replay, error handling. Essential for reliable message processing — prevent lost messages on processing failure.

Q10. What is the difference between SQS Standard and SQS FIFO?

Standard: best-effort ordering (messages may arrive out of order), at-least-once delivery (duplicates possible), nearly unlimited throughput. FIFO: strict order guaranteed within `MessageGroupId`, exactly-once within 5-minute deduplication window, 300 TPS (3000 with batching). Choose Standard for: event notifications, fan-out, high-throughput background jobs. Choose FIFO for: order processing, financial transactions where order matters.

Q11. What is SQS visibility timeout?

When a consumer receives a message, it becomes invisible to other consumers for the visibility timeout period. If consumer processes successfully → deletes message before timeout. If consumer fails/crashes → message becomes visible again after timeout for retry. Default: 30 seconds. Tune to max expected processing time + buffer. Too short: false positive retries. Too long: slow recovery after crashes. Extend via `ChangeMessageVisibility` API during long processing.

Q12. How does Kafka handle replication and fault tolerance?

Each partition has one leader and N-1 followers (replication factor). Leader handles reads and writes. Followers replicate from leader. ISR (In-Sync Replicas): replicas within configurable lag (`replica.lag.time.max.ms`). Producer `acks=all`: leader waits for all ISR to replicate before ACK. If leader fails: controller elects new leader from ISR. `min.insync.replicas`: minimum ISR count before accepting writes — prevents data loss if followers are too far behind.

Q13. What is a Kafka consumer offset and why is it important?

Offset is the position of a consumer within a partition — the ID of the last processed message. Stored in `__consumer_offsets` internal topic. Commit strategies: auto-commit (`enable.auto.commit=true`, every 5s by default — risk: commit before processing = data loss on crash), manual commit after processing (at-least-once), transactional commit with side effect (exactly-once). Reset: `auto.offset.reset=earliest` (replay from start) or `latest` (skip to now).

Q14. How does Kafka enable event replay?

Kafka retains messages for a configurable duration (`log.retention.hours`, default 168h = 7 days) or by size. Consumer can reset offset to any position: `seekToBeginning()`, `seekToEnd()`, `seek(partition, offset)`. Consumer group reset CLI: `kafka-consumer-groups.sh --reset-offsets`. Use cases: reprocess events after bug fix, bootstrap new service with historical data, audit trail, debugging. Log compaction enables indefinite retention of latest per-key state.

Q15. What is the idempotent consumer pattern?

Process each message exactly once even with at-least-once delivery. Track processed message IDs in DB or cache. Before processing: check if message_id already exists in processed_messages table. If yes: skip (already done). If no: process + insert message_id atomically. Deduplication window: only need to check recent IDs. For SQS FIFO: built-in 5-minute window. Essential when side effects are non-idempotent (charge credit card, send email).

Q16. What is Apache Pulsar and how does it differ from Kafka?

Pulsar separates compute (brokers) from storage (Apache BookKeeper). Brokers are stateless — scale independently from storage. Kafka: brokers are stateful (own partition data). Pulsar: native multi-tenancy (tenants/namespaces), tiered storage offload to S3. Supports both queuing (subscription models: exclusive, shared, key-shared) and streaming. Geo-replication built-in. More complex operations than Kafka. Growing adoption for multi-cloud/multi-tenant use cases.

Q17. How does Spring Kafka work?

@KafkaListener(topics = "orders", groupId = "payment-service") annotates consumer methods. KafkaTemplate for producing messages. @EnableKafka enables listener container. ContainerFactory: configure deserializer, error handler, retry policy (SeekToCurrentErrorHandler with backoff). Transactions: @Transactional on listener method + KafkaTransactionManager. Batch listeners: List<ConsumerRecord> parameter. Retry: DefaultErrorHandler with FixedBackOff or ExponentialBackOff.

Q18. What is a message broker vs a message bus?

Message broker (RabbitMQ, ActiveMQ): intelligent middleman that routes, transforms, and filters messages. Knows about consumers, manages delivery, ACKs. Stateful: tracks queue depth, consumer state. Message bus (Kafka): dumb pipe, fast log. Producers and consumers are decoupled from the bus itself. Bus doesn't track consumers — each manages own offset. Broker: better for complex routing logic. Bus: better for high-throughput streaming.

Q19. What is backpressure in messaging systems?

Backpressure: slow consumers signal to producers to slow down. Reactive Streams: subscriber requests N items, publisher sends at most N. Kafka: no native backpressure — consumer controls pace by controlling poll rate (max.poll.records). RabbitMQ: channel QoS (prefetch_count) limits unACKed messages per consumer — producer/broker slows when consumers are full. SQS: consumer throttles by reducing receive rate. Application-level: max queue depth triggers reject/retry.

Q20. What is the fan-out pattern with Kafka?

One Kafka topic, multiple independent consumer groups, each with different processing logic. Example: "order.placed" topic → (1) Payment service group processes payments, (2) Inventory group reserves stock, (3) Notification group sends email, (4) Analytics group updates dashboards. Each group processes at its own pace without affecting others. Zero coordination required. This is the core advantage over direct REST calls: loose coupling, independent scaling.

Q21. How do you prevent message loss in RabbitMQ?

Publisher confirms: broker ACKs after message persisted to disk. Consumer acknowledgment: basic_ack after processing (not before). Durable queues: survive broker restart (x-durable). Persistent messages: delivery_mode=2. Mirrored queues (classic HA) or quorum queues (Raft-based, preferred): replicate queue across nodes. DLX for failed messages. Lazy queues: store messages on disk early to handle large backlogs without OOM.

Q22. What is the competing consumers pattern?

Multiple consumer instances listen on the same queue — whichever is available picks up the next message. Enables parallel processing: 10 consumer instances = 10× throughput. RabbitMQ: multiple consumers on same queue, AMQP distributes round-robin. Kafka: multiple consumers in same consumer group, each assigned different partitions. SQS: multiple receivers calling `ReceiveMessage` — first to receive gets message (invisible to others). Key requirement: processing must be idempotent.

Q23. What is Kafka Streams and when do you use it?

Kafka Streams is a Java library for stream processing directly within your application — no separate cluster (Spark, Flink). DSL: `filter()`, `map()`, `flatMap()`, `groupBy()`, `aggregate()`. Stateful operations: `count()`, `aggregate()` stored in local RocksDB backed by Kafka topic. Windowing: tumbling (fixed), hopping, sliding, session windows. Exactly-once processing mode. Use when: stream processing logic belongs in your Java app, simple topology, no need for Flink/Spark cluster overhead.

Q24. How do you monitor a Kafka cluster?

JMX metrics via Prometheus JMX Exporter. Key metrics: consumer lag per partition (crucial — lag = queue depth), broker request rate, under-replicated partitions (replication lag), ISR shrinks (follower falling behind), producer throughput, consumer throughput. Tools: Confluent Control Center, Kafka UI, Burrow (lag monitoring), Grafana dashboards. Alert on: consumer lag > threshold (processing can't keep up), under-replicated partitions (data loss risk), no leader for partition (partition unavailable).

Q25. What is the AMQP protocol?

Advanced Message Queuing Protocol: open standard wire protocol for message-oriented middleware. RabbitMQ implements AMQP 0-9-1. Defines: connection (TCP), channels (virtual connections within connection), exchanges, queues, bindings. Message framing: method frames, header frames, body frames, heartbeat frames. `Basic.Publish` sends message; `Basic.Get` or `Basic.Consume` receives. `Basic.Ack/Nack` for acknowledgment. Compared to Kafka: protocol overhead per message; not designed for high-throughput streaming.

Q26. How do you implement a saga with Kafka?

Choreography-based saga: each service publishes events when local transaction completes. Next service listens on event topic. Compensating event on failure. Example: `OrderSaga` — (1) `OrderService` publishes `OrderCreated`, (2) `PaymentService` consumes, publishes `PaymentCompleted` or `PaymentFailed`, (3) `InventoryService` consumes `PaymentCompleted`, reserves stock. On failure: compensating events trigger rollback in upstream services. No central coordinator — each service reacts to events.

Q27. What is the difference between push and pull in messaging?

Push (RabbitMQ `basic.consume`, AMQP): broker actively sends messages to consumer as they arrive. Fast delivery, consumer may be overwhelmed without prefetch limit. Pull (Kafka, SQS): consumer polls for messages when ready. Consumer controls pace — natural backpressure. Kafka: `poll()` with `max.poll.records` limit. SQS: `ReceiveMessage` with `WaitTimeSeconds` for long polling. Push best for: low-latency real-time. Pull best for: batch processing, backpressure control.

Q28. How does Kafka handle partition rebalancing?

Rebalancing triggered when: consumer joins/leaves group, topic partition count changes, `session.timeout.ms` exceeded (consumer missed heartbeat). Group coordinator (broker) assigns partitions to consumers. Eager rebalancing (default): all consumers stop, partitions revoked, new assignments made — brief pause in processing. Cooperative rebalancing: only affected partitions revoked — no global pause. Spring Kafka 2.4+: `CooperativeStickyAssignor` for incremental rebalancing.

Q29. What is the difference between at-least-once and exactly-once processing?

At-least-once: message processed one or more times. On failure/crash before offset commit: message reprocessed. Acceptable when side effects are idempotent (e.g., upsert by ID). Exactly-once: message processed exactly once. Requires: idempotent operations OR distributed transactions (Kafka transactions) committing offset and side effect atomically. Implementation overhead: 2-5× performance cost. Use for: financial transactions, inventory updates, any non-idempotent operation.

Q30. How do you handle poison pill messages in Kafka?

Poison pill: message that always fails processing (malformed data, unexpected format, business rule violation). Without handling: consumer retries forever, partition consumption stops (lag grows). Solutions: (1) `DeadLetterPublishingRecoverer` (Spring Kafka): after N retries, publish to .DLT (dead-letter topic). (2) `SeekToCurrentErrorHandler`: skip after max retries. (3) Log and skip: alert + manual intervention. DLT allows: inspection, fix, replay. Never allow one bad message to block entire partition.

Q31. What is Kafka's ISR (In-Sync Replicas)?

ISR: set of replicas that are fully caught up with the leader (within `replica.lag.time.max.ms` = 30s default). `acks=all`: producer waits for all ISR to ACK. Replica falls out of ISR if it lags too far behind (network partition, slow disk). `min.insync.replicas` (default 1): minimum ISR size before accepting writes. `min.insync.replicas=2` with `RF=3`: tolerates 1 failure, refuses writes if only 1 replica in sync — prevents data loss at cost of availability.

Q32. How do you scale Kafka consumers?

Consumer throughput limited by: partition count (max parallel consumers = partition count), processing time per record. To scale: increase partition count (irreversible, plan ahead), add consumers (up to partition count), parallelize processing within consumer (async handlers), increase `max.poll.records` (batch size). Consumer lag alert → trigger auto-scaling of consumer pods. KEDA (Kubernetes Event-Driven Autoscaling): scale pods based on Kafka consumer lag — native integration with Kafka metrics.

Q33. What is a Kafka connector and Kafka Connect?

Kafka Connect: framework for streaming data between Kafka and external systems without custom code. Source connector: external system → Kafka (e.g., Debezium CDC connector reads MySQL binlog → publishes change events to Kafka). Sink connector: Kafka → external system (e.g., JDBC sink connector → PostgreSQL, Elasticsearch sink connector → ES). Distributed mode: scalable, fault-tolerant. Standalone mode: single process for development. Eliminates custom ETL pipelines for common integrations.

Q34. How does RabbitMQ quorum queues improve on classic mirrored queues?

Classic mirrored queues (HA): all-or-nothing mirroring, lacks strong consistency guarantees, deprecated in RabbitMQ 3.9+. Quorum queues (Raft-based): elected leader with followers, majority ACK before message confirmed, safe under network partitions (CP). Benefits: predictable behavior under failure, no "mirror promotion" inconsistency, better performance for persistent messages. Trade-off: minimum 3 nodes required, slightly higher write latency than single-node classic queues.

Q35. What is the outbox pattern for reliable event publishing?

Problem: saving to DB and publishing to Kafka are two separate operations — one can fail. Solution: write event to outbox table in same DB transaction as business data. Separate outbox processor reads outbox table and publishes to Kafka, deletes row on success. Transactional Outbox: atomic DB write + guaranteed eventual Kafka publish. CDC-based: Debezium reads DB binlog/WAL and publishes outbox table changes to Kafka automatically. Guarantees: events published if and only if DB transaction committed.

Q36. How do you implement request-reply over a message queue?

Request-reply: synchronous request pattern over async infrastructure. Requestor sends message with `correlation_id` and `reply_to` queue header. Handler processes and sends response to `reply_to` queue with same `correlation_id`. Requestor blocks (or uses `CompletableFuture`) waiting for response. Spring AMQP: `RabbitTemplate.convertSendAndReceive()` handles this automatically. Use for: synchronous orchestration where you need the result but want loose coupling and broker-based reliability.

Q37. How do you choose between Kafka and RabbitMQ?

Choose Kafka when: event streaming at high throughput (millions/sec), event replay needed, multiple independent consumers of same events, event sourcing, audit log, data pipeline, real-time analytics (Kafka Streams). Choose RabbitMQ when: complex routing logic (topic/headers exchange), task queues with acknowledgment, RPC over queues, AMQP compliance required, smaller scale, priority queues. Both support: pub/sub, persistence, clustering. Kafka is not a drop-in replacement for RabbitMQ's routing flexibility.

Q38. What is message deduplication in SQS FIFO?

SQS FIFO provides exactly-once delivery within 5-minute deduplication window. Two methods: Content-based: SHA-256 of message body as dedup ID — identical bodies deduplicated. `MessageDeduplicationId`: provide custom ID per message (e.g., `order_id`). If same dedup ID received within 5 minutes: second message silently discarded. `MessageGroupId`: defines ordering group — messages in same group strictly ordered. Different groups processed concurrently (up to 20,000 distinct groups).

Q39. What is event-driven architecture and how do message queues enable it?

EDA: services communicate by publishing and subscribing to events rather than direct API calls. Services react to events asynchronously. Benefits: loose coupling (publisher doesn't know subscribers), resilience (subscriber failure doesn't affect publisher), scalability (add subscribers without changing publisher), audit trail (event log). Message queues/Kafka enable EDA by: guaranteeing delivery, decoupling timing (producer/consumer run independently), supporting multiple subscribers per event type.